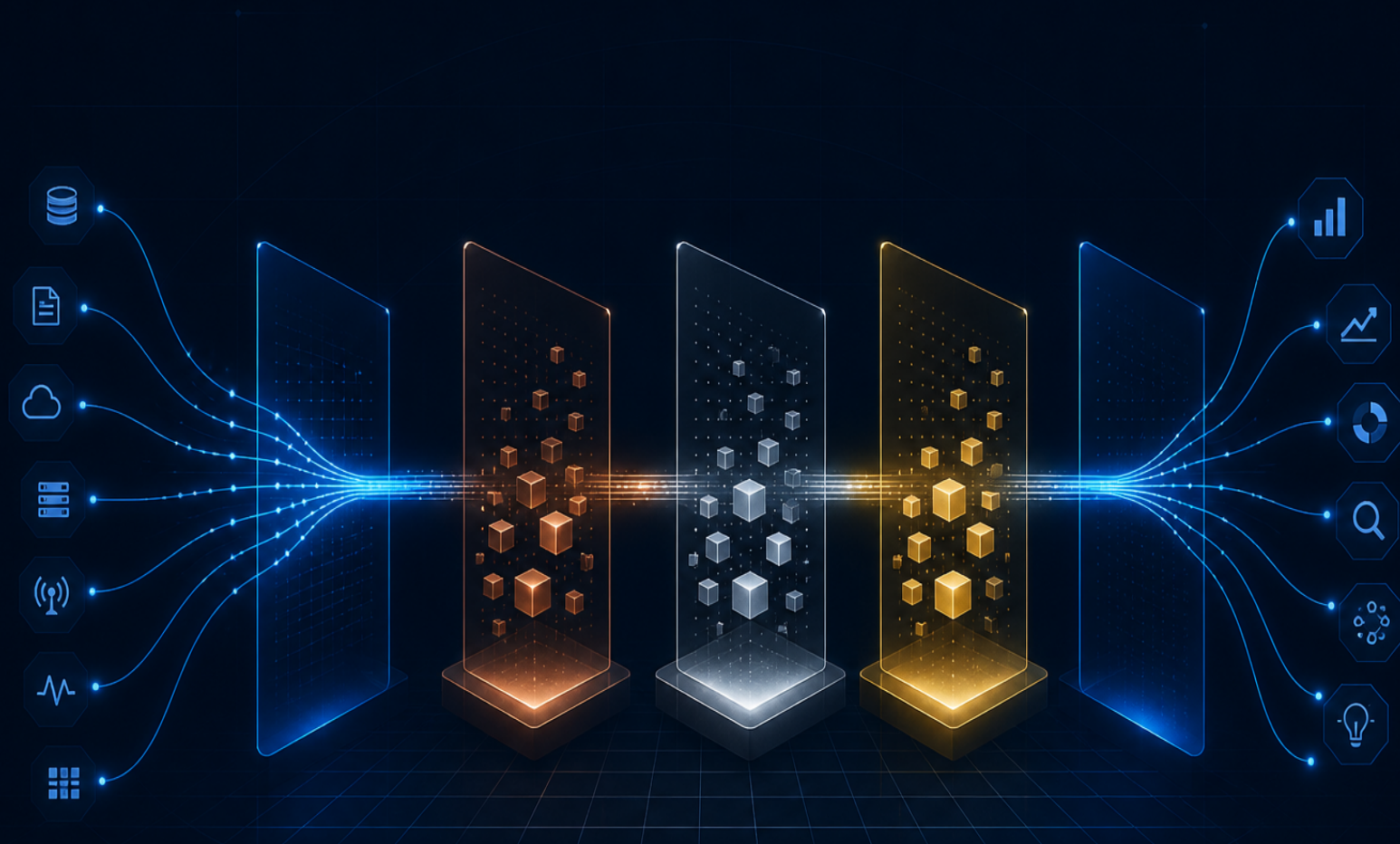




Operator ETL

Agentic Data Platform — Engineering White Paper

v2.1 · Architecture · MCP · GCP · NFRs · ADRs

Table of Contents

- Agentic Data Intake, Warehouse, and Insights
 - White Paper — Architecture, MCP Tool Surface, and GCP Implementation
 - Scope
 - Implementation status legend
 - Source-of-truth files (IMPLEMENTED)
- 1. Introduction
 - 1.1 The problem
 - 1.2 The Operator ETL answer
- 2. Engineering requirements (NFRs)
 - 2.1 Functional NFRs
 - 2.2 Agentic NFRs (v2)
 - 2.3 Performance and throughput
 - 2.4 Recovery objectives
- 3. Architecture Decision Records
 - ADR-001: Medallion architecture (bronze / silver / gold)
 - ADR-002: LangGraph over implicit agent swarm
 - ADR-003: MCP as the agent boundary
 - ADR-004: Fail-closed quality gate
 - ADR-005: DuckDB local → BigQuery production
- 4. Design principles
- 5. System architecture
 - 5.1 End-to-end flow
 - 5.2 Deterministic vs agentic
- 6. Data contracts and schema evolution
 - 6.1 Bronze contract
 - 6.2 Silver contract ([SilverOrder](#))
 - 6.3 Quarantine contract
 - 6.4 Gold contract — insight citation allowlist
 - 6.5 Schema evolution policy
- 7. Failure modes and error taxonomy
- 8. Control plane — LangGraph
 - 8.1 Why LangGraph
 - 8.2 Pipeline state (no PII values)
 - 8.3 Graph compile (pseudocode)
 - 8.4 Retry policy per node
 - 8.5 Critic algorithm (deterministic)
 - 8.6 HITL resume
 - 8.7 Conditional edges
 - 8.8 Checkpoint storage
- 9. Policy plane — PII and budgets
 - 9.1 PII handling pipeline
 - 9.2 Vault
 - 9.3 Budgets (per run)
- 10. Data plane — medallion warehouse

- 10.1 Layers
- 10.2 Idempotency
- 10.3 Quality gate
- 11. MCP tool surface
 - 11.1 Architecture
 - 11.2 Transport
 - 11.3 Tool catalog with I/O schemas
 - 11.4 MCP error responses
 - 11.5 SQL allowlist spec ([sql/allowlist.yaml](#))
 - 11.6 Cursor registration
- 12. GCP implementation
 - 12.1 Target architecture
 - 12.2 Service mapping
 - 12.3 IAM matrix (least privilege)
 - 12.4 Threat model (STRIDE-lite)
 - 12.5 Cloud Run [graph-runner](#) spec
 - 12.6 BigQuery datasets
 - 12.7 Cloud Build pipeline
- 13. Reliability patterns
- 14. Observability, SLIs, and runbooks
 - 14.1 SLIs
 - 14.2 SLOs (proposed)
 - 14.3 Structured log schema
 - 14.4 Runbooks
- 15. Testing strategy
- 16. Operational model
- 17. Implementation roadmap
- 18. Conclusion
- Appendix A — Demo walkthrough
- Appendix B — MCP + GCP quick reference
- Appendix C — References

Abstract

Most “AI ETL” demos fail in production because they let a language model write SQL against raw data, skip validation, leak PII into traces, and produce insights that cannot be audited. **Operator ETL** takes a different approach: a **deterministic data plane** (bronze → silver → gold) does the actual ETL, while a **LangGraph control plane** orchestrates decisions, and a **policy plane** enforces PII redaction and tool budgets before any model call.

Agents do not replace ETL. They call **allowlisted MCP tools** that wrap the data plane. Insights are generated from gold aggregates only, then verified by a **critic node** that rejects any number not present in the warehouse. The system is **checkpointed, resumable, and fail-closed** on quality and privacy.

1. Introduction

IMPLEMENTE D

Status: IMPLEMENTED (data plane + agentic layers local) · PARTIAL (GCP lift)

1.1 The problem

Teams want to “drop data in and get insights.” The naive approach — a chatbot with database access — breaks in five predictable ways:

1. **Nondeterminism** — the same file produces different SQL or counts on re-run.
2. **PII leakage** — emails and account numbers appear in prompts, logs, and traces.
3. **Silent bad data** — invalid rows load into metrics without quarantine.
4. **Hallucinated insights** — narrative text cites numbers that do not exist in the warehouse.
5. **No recovery** — a crash mid-run restarts from scratch; no audit trail.

1.2 The Operator ETL answer

[illegible]

Core claim: LLMs decide **whether** to map a column or **how** to phrase an insight. Python and SQL decide **what data exists**.

2. Engineering requirements (NFRs)

Status: PARTIAL — v1 NFRs measured; v2 NFRs specified

2.1 Functional NFRs

NFR	Target	v1 status	Rationale
Ingest idempotency	100% dedupe on same content hash	Met — test_ingest_is_idempotent	At-least-once delivery safe
Row-level validation	100% invalid rows quarantined, not silver	Met — test_quarantine_in_valid_rows	No silent bad data
Pipeline determinism (data plane)	Bit-identical gold given same bronze	Met — pure SQL marts	Replay / audit
Quality fail-closed	KPIs withheld when gate fails	Met — test_quality_gate_blocks	Operator trust
Source extensibility	New source = registry entry only	Met — pipelines/demo.yaml	No pipeline rewrite

2.2 Agentic NFRs (v2)

NFR	Target	Rationale
PII in LLM context	Zero raw PII values	Fail-closed policy
Insight faithfulness	100% numeric citations ∈ gold_metrics	Critic gate
Checkpoint recovery	Resume within 1 invoke after crash	LangGraph HITL
Tool denial	100% reject for vault_decrypt , raw SQL	MCP boundary
Runaway prevention	Budget exceeded → failed within 1 node	No infinite loops

2.3 Performance and throughput

Assumption	Value
Processing model	Batch (file/API pull), not streaming
v1 local benchmark	21-row demo CSV < 2s end-to-end
v1 target	< 30s for 10k-row CSV on laptop (DuckDB)
v2 Cloud Run	900s timeout; concurrency=1 per run
Expected ingest rate	1–50 files/hour (operator-driven), burst via Pub/Sub

2.4 Recovery objectives

Metric	Local (v1)	GCP (v2)
RPO (checkpoint)	N/A (single-shot CLI)	Last completed graph node
RTO (resume)	Re-run etl run (hash skip)	graph.invoke resume < 60s

Metric	Local (v1)	GCP (v2)
Audit retention	<code>pipeline_runs</code> indefinite in DuckDB	BQ audit table 90d

3. Architecture Decision Records

SPECIFIED

Status: SPECIFIED — engineering rationale for reviewers

ADR-001: Medallion architecture (bronze / silver / gold)

Status	Accepted — IMPLEMENTED (v1)
Context	Operators need raw audit trail, typed analytics rows, and aggregate KPIs without mixing concerns
Decision	Three layers: bronze (immutable JSON), silver (Pydantic-validated), gold (SQL marts)
Alternatives rejected	Single-table CSV load (no quarantine path); streaming-only (overkill for v1)
Consequences	(+) Clear replay path, testable layers; (–) More tables to manage

ADR-002: LangGraph over implicit agent swarm

Status	Accepted — SPECIFIED (v2)
Context	Agent systems need explicit control flow, checkpoints, and HITL for production
Decision	LangGraph <code>StateGraph</code> with typed <code>PipelineState</code> , conditional edges, <code>PostgresSaver</code>
Alternatives rejected	CrewAI/AutoGen chat loops (implicit flow); Airflow+LLM (wrong abstraction for agent decisions)
Consequences	(+) Replayable, interview-friendly; (–) Graph boilerplate upfront

ADR-003: MCP as the agent boundary

Status	Accepted — SPECIFIED (v2)
Context	Agents need typed, auditable access to data plane without raw SQL
Decision	Operator ETL MCP server with 10 allowlisted tools; Pydantic args; structured errors
Alternatives rejected	Python imports inside prompts; generic <code>execute_sql</code> tool
Consequences	(+) Same tools local (stdio) and cloud (SSE); (–) MCP server to maintain

ADR-004: Fail-closed quality gate

Status	Accepted — IMPLEMENTED (v1)
Context	Dashboards that show wrong KPIs destroy operator trust faster than empty dashboards
Decision	Withhold KPIs when quarantine rate > 35%, freshness > 7 days, or silver empty
Alternatives rejected	Show KPIs with warning banner (operators ignore warnings)
Consequences	(+) Trustworthy outputs; (–) Operators must fix upstream before insights

ADR-005: DuckDB local → BigQuery production

Status	Accepted — PARTIAL
Context	Developers need zero-infra local runs; production needs scale and IAM
Decision	DuckDB file locally; BigQuery datasets in GCP; same SQL marts, dialect-adjusted
Alternatives rejected	Postgres-only from day one (local setup friction); BigQuery-only (slow dev loop)
Consequences	(+) Fast iteration + production path; (–) SQL dialect testing in CI

4. Design principles

IMPLEMENTED

Status: IMPLEMENTED (data plane principles) · SPECIFIED (agentic principles)

Principle	What it means
Agents orchestrate; ETL executes	Models never run unconstrained SQL on raw rows
Fail closed	Ambiguous PII or failed quality gate → block or escalate to human
Idempotent intake	SHA-256 file hash prevents double-loads
Typed state	Graph state holds metadata and aggregates, not PII values
Tool allowlists	Each agent node gets a fixed MCP tool set with Pydantic args
Faithfulness by construction	Critic node rejects insights with uncited numbers
Checkpoint everything	Resume from last node after crash or HITL pause

5. System architecture

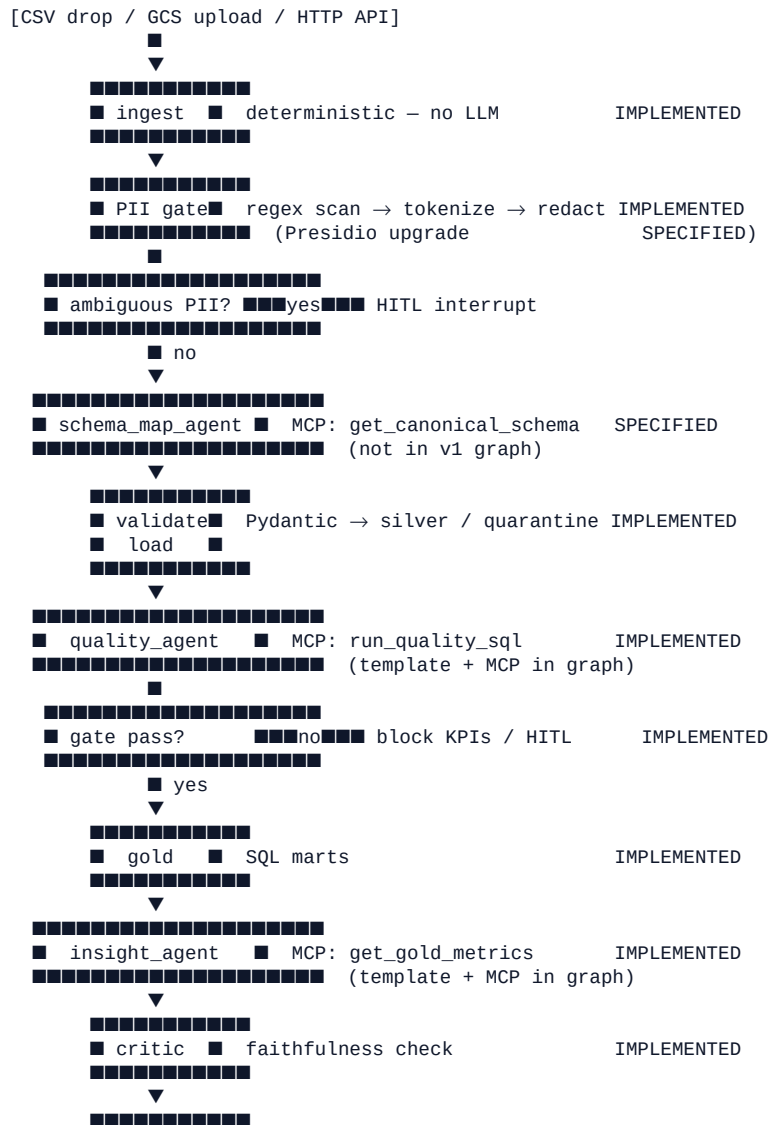
IMPLEMENTED



Fig — 5. System architecture

Status: IMPLEMENTED (v1 graph) · PARTIAL (GCP lift, Presidio, LLM nodes)

5.1 End-to-end flow



IMPLEMENTED

Step	Type	v1 status
File/API extract	Deterministic	IMPLEMENTED
Bronze load	Deterministic	IMPLEMENTED
PII scan/tokenize	Deterministic (+ HITL if ambiguous)	IMPLEMENTED (regex); Presidio SPECIFIED
Schema mapping	Agent	SPECIFIED (not in v1 graph)
Validate/load silver	Deterministic	IMPLEMENTED
Quality diagnosis	Agent	IMPLEMENTED (template + MCP)
Gold SQL	Deterministic	IMPLEMENTED
Insight narrative	Agent	IMPLEMENTED (template + MCP); LLM SPECIFIED
Critic	Deterministic rules	IMPLEMENTED

IMPLEMENTE

Column	Type	Required	Description
payload	JSON	yes	Original row as JSON object
_content_hash	VARCHAR	yes	SHA-256 of source file bytes
_file_name	VARCHAR	yes	Source filename
_source	VARCHAR	yes	Registry source name (demo , inbox , http)
_ingested_at	TIMESTAMP	yes	UTC ingestion time
_row_num	INTEGER	yes	1-based row index within file

```
class SilverOrder(BaseModel):
    order_id: str = Field(min_length=1)
    customer_id: str = Field(min_length=1)
    ordered_at: datetime
```

```

amount: float = Field(gt=0)
sku: str = Field(min_length=1)
status: str = Field(min_length=1)

```

Validators: strip whitespace on strings; parse \$ and , from amount strings.

6.3 Quarantine contract

Column	Description
<code>payload</code>	Original row JSON
<code>error</code>	Machine-readable reason (Pydantic message or custom)

Demo quarantine examples (from `samples/orders.csv`, verified in tests):

Row issue	Error class	Example error
Empty <code>order_id</code>	<code>ROW_VALIDATION</code>	missing fields / empty
<code>ordered_at = not-a-date</code>	<code>ROW_VALIDATION</code>	Input should be a valid datetime
<code>amount = -5.00</code>	<code>ROW_VALIDATION</code>	Input should be greater than 0
<code>amount = free</code>	<code>ROW_VALIDATION</code>	valid number parse failure
Duplicate <code>order_id</code>	<code>DUPLICATE_ORDER_ID</code>	duplicate order_id ORD-xxx

Demo counts: **21 bronze** → **17 silver**, **4 quarantined (19.1%)**.

6.4 Gold contract — insight citation allowlist

Insight agent may cite **only** these metric keys:

Mart	Allowed keys
<code>gold_kpis</code>	<code>order_count</code> , <code>customer_count</code> , <code>revenue</code> , <code>avg_order</code> , <code>latest_order_at</code> , <code>freshness_at</code>
<code>gold_volume_daily</code>	<code>order_date</code> , <code>orders</code> , <code>revenue</code> (aggregates only)
<code>gold_top_skus</code>	<code>sku</code> , <code>orders</code> , <code>revenue</code> (top-N only)
<code>gold_quality</code>	<code>bronze_rows</code> , <code>silver_rows</code> , <code>quarantined_rows</code> , <code>quarantine_rate</code> , <code>last_ingest_at</code>

Critic rejects any number in narrative not found in serialized `gold_metrics` dict.

6.5 Schema evolution policy

Change type	Policy
Add optional silver column	Additive; default null; update Pydantic model
Rename column	New source registry entry + contract version bump
Breaking type change	New silver table or <code>silver_orders_v2</code> ; never silent migrate
New gold mart	New SQL file in <code>sql/marts/</code> ; add keys to citation allowlist

7. Failure modes and error taxonomy

IMPLEMENTED

Status: IMPLEMENTED (row validation, quality gate) · SPECIFIED (DLQ, PII, critic)

ExtractFailure ■■■■■■■■■■ DLQ (after 3 retries)
 PIIAmbiguous ■■■■■■■■■■ HITL interrupt
 RowValidationFail ■■■■■■■■ quarantine_orders
 DuplicateOrderId ■■■■■■■■ quarantine_orders
 QualityGateFail ■■■■■■■■ block KPIs (insights withheld)
 CriticFail ■■■■■■■■■■ revise (max 2) → HITL
 BudgetExceeded ■■■■■■■■■■ status=failed

Error class	HTTP/API	Handling	Operator action
EXTRACT_HTTP_4XX	4xx	Retry 3x exp backoff → DLQ	Fix URL / credentials
EXTRACT_HTTP_5XX	5xx	Retry 3x → DLQ	Check upstream API
ROW_VALIDATION	—	Quarantine row; continue run	Fix upstream CSV
DUPLICATE_ORDER_ID	—	Quarantine row	Dedupe source file
QUALITY_GATE	—	Block KPIs; show quality panel	Investigate quarantine rate
PII_AMBIGUOUS	—	HITL interrupt	Approve column classification
PII_BLOCKED	—	status=failed	Remove or tokenize column
CRITIC_VIOLATION	—	Revise ≤2 → HITL	Review insight draft
BUDGET_EXCEEDED	—	status=failed	Increase budget or simplify run
CHECKPOINT_CORRUPT	—	Fail; manual replay from bronze	Restore Cloud SQL backup

Terminal run states: [complete](#), [failed](#), [needs_human](#).

8. Control plane — LangGraph

SPECIFIED

Status: SPECIFIED — v1 CLI pipeline mirrors ingest→transform→gold without graph

8.1 Why LangGraph

- **Explicit graph** — nodes and conditional edges, not an implicit loop
- **Typed state** — `PipelineState` is the contract between nodes
- **Checkpoints** — `PostgresSaver` (GCP) or `SqliteSaver` (local)
- **Human-in-the-loop** — `interrupt()` before persist when critic score is low
- **Time-travel** — replay a run for debugging and evals

8.2 Pipeline state (no PII values)

```
class PipelineState(TypedDict):
    run_id: str
    source: str
    artifact_uri: str          # gs://bucket/path or local path
    content_hash: str
    pii_findings: list[PiiFinding]  # column, type, count – NOT values
    vault_ref: str
    schema_proposal: dict | None
```

```

bronze_table: str | None
quality_report: dict | None
gold_metrics: dict | None # aggregates only
insight_draft: str | None
critic: dict | None # score, violations, cited_metric_keys
status: Literal["running", "needs_human", "failed", "complete"]
errors: Annotated[list[str], add]

```

State reducers: `errors` uses `operator.add` (append); all other fields last-write-wins.

8.3 Graph compile (pseudocode)

```

graph = StateGraph(PipelineState)
graph.add_node("ingest", ingest_node)
graph.add_node("pii_gate", pii_gate_node)
graph.add_node("schema_map_agent", schema_map_node)
graph.add_node("validate_load", validate_load_node)
graph.add_node("quality_agent", quality_node)
graph.add_node("build_gold", build_gold_node)
graph.add_node("insight_agent", insight_node)
graph.add_node("critic", critic_node)
graph.add_node("persist", persist_node)

graph.set_entry_point("ingest")
graph.add_conditional_edges("pii_gate", route_pii, {...})
graph.add_conditional_edges("quality_agent", route_quality, {...})
graph.add_conditional_edges("critic", route_critic, {...})

app = graph.compile(checkpointer=PostgresSaver(conn), interrupt_before=["persist"])

```

8.4 Retry policy per node

Node	Retries	Backoff	On exhaustion
HTTP extract	3	exp 1s, 2s, 4s	DLQ
LLM nodes (schema, quality, insight)	2	exp 2s, 4s	failed
Deterministic (ingest, validate, gold)	0	—	failed
Critic revision loop	2	—	HITL

8.5 Critic algorithm (deterministic)

```

def critic_check(insight_draft: str, gold_metrics: dict) -> CriticResult:
    numbers = extract_numeric_tokens(insight_draft)
    allowed = flatten_numeric_values(gold_metrics)
    violations = [n for n in numbers if not fuzzy_match(n, allowed)]
    return CriticResult(passed=len(violations) == 0, violations=violations)

```

`fuzzy_match` tolerates rounding to 2 decimal places only — no fabricated precision.

8.6 HITL resume

```

config = {"configurable": {"thread_id": run_id}}
state = app.get_state(config)
app.update_state(config, {"approved": True}) # operator approval
app.invoke(None, config) # resume from interrupt

```

8.7 Conditional edges

From	Condition	To
<code>pii_gate</code>	high-confidence clean	<code>schema_map_agent</code>

From	Condition	To
pii_gate	ambiguous field	needs_human
pii_gate	blocked pattern	failed
quality_gate	pass	build_gold
quality_gate	high quarantine rate	needs_human
critic	all numbers cited	persist
critic	violation + retries left	insight_agent
critic	violation + exhausted	needs_human

8.8 Checkpoint storage

Environment	Checkpointers	Path
Local dev	SqliteSaver	warehouse/checkpoints.db
GCP prod	PostgresSaver	Cloud SQL PostgreSQL 15

Same `thread_id = run_id` resumes exactly where the run stopped.

9. Policy plane — PII and budgets

SPECIFIED

Status: SPECIFIED

9.1 PII handling pipeline

1. **Scan** — Microsoft Presidio (EMAIL, PHONE, PERSON, US_SSN, etc.)
2. **Tokenize** — AES-encrypted vault; token format `EMAIL_0x{ab}`
3. **Redact** — all LLM-facing views use tokens only
4. **Gold exclusion** — PII columns dropped or bucketed before marts
5. **Trace filter** — Langfuse handler strips value patterns

Confidence thresholds:

Presidio score	Action
≥ 0.85	Auto-tokenize
0.40 – 0.84	HITL interrupt
< 0.40 (non-PII)	Pass through

9.2 Vault

Store	Contents	MCP access
pii_vault	token → encrypted original	Denied

Store	Contents	MCP access
Graph state	findings metadata (no values)	Read by policy nodes
Gold marts	aggregates	get_gold_metrics only

9.3 Budgets (per run)

Cap	Default	Exceeded →
LLM calls	12	status=failed
Tokens	24,000	status=failed
MCP tool calls	30	status=failed

10. Data plane — medallion warehouse

IMPLEMENTED

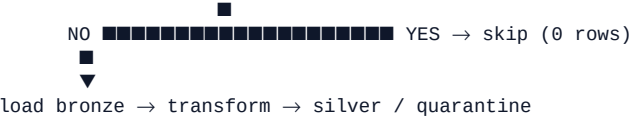
Status: IMPLEMENTED

10.1 Layers

Layer	Table	Contents
Bronze	bronze_raw	Raw JSON + lineage columns
Silver	silver_orders	Pydantic-validated rows
Quarantine	quarantine_orders	Rejected rows + error
Gold	gold_kpis , gold_volume_daily , gold_top_skus , gold_quality	SQL aggregates
Ops	ingest_files , pipeline_runs	Idempotency + audit

10.2 Idempotency

New file → SHA-256 → exists in ingest_files?



Implemented in `load_bronze()` + `already_ingested()` — see `src/operator_etl/load/duckdb.py`.

10.3 Quality gate

Check	Default	Env var	On failure
Quarantine rate	≤ 35%	OPERATOR_ETL_MAX_QUARANTINE_RATE	KPIs withheld
Freshness	≤ 7 days	OPERATOR_ETL_MAX_FRESHNESS_HOURS (168)	KPIs withheld

Check	Default	Env var	On failure
Silver rows	> 0	—	KPIs withheld

Implemented in `quality_gate()` — `src/operator_etl/insights/metrics.py`.

11. MCP tool surface

SPECIFIED

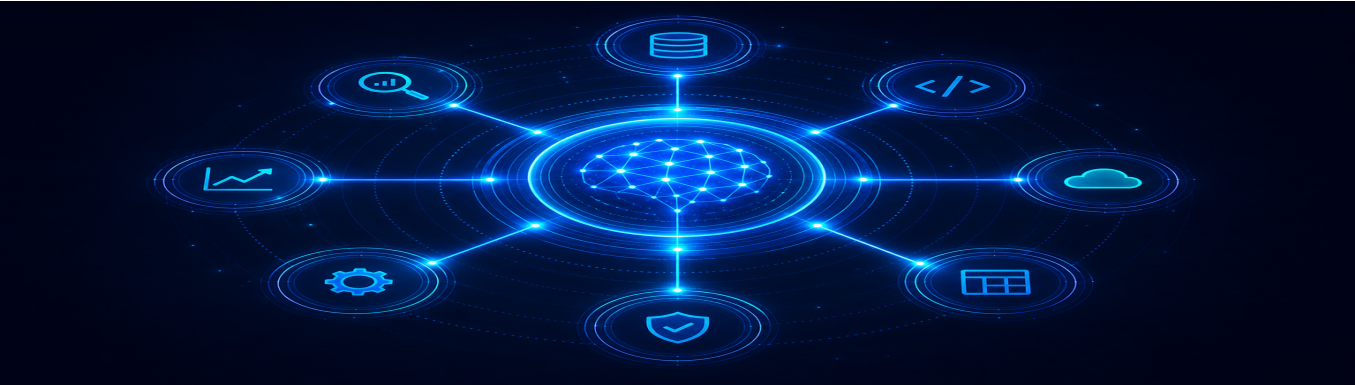


Fig — 11. MCP tool surface

Status: SPECIFIED — tool functions mirror v1 Python APIs

11.1 Architecture

Cursor agent ■■■MCP stdio■■■ operator-etl-mcp ■■■ policy check ■■■ data plane
Cloud agent ■■■MCP SSE■■■ operator-etl-mcp (Cloud Run)
LangGraph node ■■■in-process■■■ same tool functions (no MCP hop)

11.2 Transport

Transport	When	Client
stdio	Local dev, Cursor	.cursor/mcp.json
SSE	GCP Cloud Run	Remote agents, dashboard

11.3 Tool catalog with I/O schemas

`list_pending_artifacts`

- **Input:** { "source": "inbox" }
- **Output:** { "artifacts": [{ "uri", "name", "size_bytes" }] }
- **Idempotent:** yes

`ingest_artifact`

- **Input:** { "uri": "gs://... or path", "source": "inbox" }
- **Output:** { "content_hash", "rows_in", "skipped": false }
- **Idempotent:** yes (hash dedupe)

`get_canonical_schema`

- **Input:** { "contract": "SilverOrder" }

- **Output:** Pydantic JSON schema
- **Idempotent:** yes

`get_redacted_preview`

- **Input:** { "run_id", "limit": 3 }
- **Output:** { "columns": [...], "rows": [[tokenized values]] }
- **Denied:** vault decrypt

`propose_schema_mapping`

- **Input:** { "run_id", "mapping": { "src_col": "dest_col" } }
- **Output:** { "accepted": true, "validation_errors": [] }

`run_quality_sql`

- **Input:** { "query_id": "null_rate_by_column" } — must exist in allowlist
- **Output:** { "columns": [...], "rows": [...] }
- **Denied:** arbitrary SQL

`get_gold_metrics`

- **Input:** { "run_id" }
- **Output:** { "order_count": 17, "revenue": 1373.82, ... }
- **Denied:** bronze/silver access

`get_metric_definition`

- **Input:** { "metric_key": "revenue" }
- **Output:** { "description", "sql_ref": "sql/marts/01_gold_kpis.sql" }

`persist_insight`

- **Input:** { "run_id", "text", "critic_passed": true }
- **Output:** { "insight_id" }
- **Denied:** if `critic_passed` is false

`get_run_status`

- **Input:** { "run_id" }
- **Output:** { "status", "rows_in", "rows_silver", "rows_quarantined", "error" }

11.4 MCP error responses

```
{
  "error": "TOOL_DENIED",
  "reason": "run_quality_sql: query_id not in allowlist",
  "run_id": "abc-123"
}
```

Never return raw stack traces to agents.

11.5 SQL allowlist spec (`sql/allowlist.yaml`)

Status: SPECIFIED — file not yet in repo

```
queries:
- id: quarantine_summary
  sql: |
    SELECT error, COUNT(*) AS n
    FROM quarantine_orders
```

```

GROUP BY 1 ORDER BY 2 DESC
allowed_nodes: [quality_agent]

- id: freshness_check
  sql: |
    SELECT last_ingest_at, quarantine_rate
    FROM gold_quality
    allowed_nodes: [quality_agent]

- id: null_rate_silver
  sql: |
    SELECT
      SUM(CASE WHEN customer_id IS NULL THEN 1 ELSE 0 END) * 1.0 / COUNT(*) AS null_rate
    FROM silver_orders
    allowed_nodes: [quality_agent]

```

11.6 Cursor registration

```

{
  "mcpServers": {
    "operator-etl": {
      "command": "uv",
      "args": ["run", "operator-etl-mcp"],
      "cwd": "/path/to/operator-etl"
    }
  }
}

```

12. GCP implementation

SPECIFIED



Fig — 12. GCP implementation

Status: SPECIFIED — aligns with AI Operator project [the-ai-operator](#)

12.1 Target architecture

Cloud Scheduler ■■■ Cloud Run (graph-runner) ■■■ Cloud SQL (checkpoints)
 GCS inbox ■■■ Pub/Sub ■■■ Cloud Run (ingest trigger)
 Cloud Run (operator-etl-mcp) ■■■ BigQuery (gold read)
 Secret Manager ■■■ all services (API keys, vault key)

12.2 Service mapping

Concern	GCP service	Role
File landing	Cloud Storage gs://operator-etl-inbox/	Drop zone
Event trigger	Pub/Sub	OBJECT_FINALIZE → Cloud Run

Concern	GCP service	Role
Graph execution	Cloud Run graph-runner	LangGraph, 900s timeout
MCP endpoint	Cloud Run operator-etl-mcp	SSE for remote agents
Warehouse	BigQuery	etl_bronze , etl_silver , etl_gold datasets
Checkpoints	Cloud SQL PostgreSQL 15	LangGraph PostgresSaver
Secrets	Secret Manager	Vault key, LLM keys
Scheduling	Cloud Scheduler	Nightly freshness
CI/CD	Cloud Build + Artifact Registry	Build → deploy
Logs	Cloud Logging	Structured JSON, run_id label
Traces	Langfuse	PII-filtered spans

12.3 IAM matrix (least privilege)

Service account	GCS inbox	BigQuery	Secret Manager	Cloud SQL	Pub/Sub
etl-ingest@	objectViewer	—	—	—	subscriber
graph-runner@	objectViewer	dataEditor (etl_* datasets)	secretAccessor	client	publisher
etl-mcp@	—	dataViewer (etl_gold only)	—	—	—
cloudbuild@	—	—	—	—	—

No shared service account across ingest and MCP read paths.

12.4 Threat model (STRIDE-lite)

Threat	Mitigation
Spoofing	SA per service; no long-lived keys in images
Tampering	Bronze append-only; gold rebuilt from silver
Repudiation	pipeline_runs audit log + Cloud Logging
Information disclosure	PII vault; MCP deny list; trace redaction
Denial of service	Budget caps; Cloud Run max instances=10
Elevation of privilege	No vault_decrypt tool; IAM scoped to dataset

12.5 Cloud Run [graph-runner](#) spec

Setting	Value	Reason
CPU	2	Presidio + Pydantic
Memory	4 GiB	Graph state + BQ client
Timeout	900s	Large file ingest

Setting	Value	Reason
Concurrency	1	State isolation per run
Min / max instances	0 / 10	Cost vs burst

Environment (from Secret Manager): [OPENAI_API_KEY](#), [PII_VAULT_KEY](#), [CHECKPOINT_DATABASE_URL](#), [GCS_INBOX_BUCKET](#), [BQ_DATASET](#).

12.6 BigQuery datasets

Dataset	Table	Partition
etl_bronze	raw_events	_ingested_at DAY
etl_silver	orders	ordered_at DAY
etl_quarantine	rejected	_ingested_at DAY
etl_gold	kpis , volume_daily , top_skus , quality	none (small)

12.7 Cloud Build pipeline

steps:

- name: gcr.io/cloud-builders/docker
args: ["build", "-t", "\$_AR_HOST/operator-etl:\$SHORT_SHA", "."]
- name: gcr.io/cloud-builders/docker
args: ["push", "\$_AR_HOST/operator-etl:\$SHORT_SHA"]
- name: gcr.io/google.com/cloudsdktool/cloud-sdk
args: ["run", "deploy", "graph-runner",
"--image", "\$_AR_HOST/operator-etl:\$SHORT_SHA",
"--region", "us-central1"]

13. Reliability patterns

IMPLEMENTED

Status: IMPLEMENTED (idempotency, quality fail-closed) · SPECIFIED (DLQ, saga, HITL)

Pattern	Implementation
Idempotent ingest	SHA-256 in ingest_files
At-least-once events	Pub/Sub redelivery safe via hash dedupe
Saga-lite	Silver survives gold failure; replay from checkpoint
Dead letter	dlq_events + Pub/Sub DLQ after 5 retries
HITL resume	graph.update_state() + invoke(None)
Critic loop cap	Max 2 revisions → HITL
Quality fail-closed	No KPIs if gate failed

14. Observability, SLIs, and runbooks

Status: PARTIAL — [pipeline_runs](#) IMPLEMENTED; Cloud Monitoring SPECIFIED

14.1 SLIs

SLI	Formula
gate_pass_rate	runs passing quality gate / total runs
quarantine_rate_p95	p95 of gold_quality.quarantine_rate over 7d
ingest_latency_p95	p95 of finished_at - started_at in pipeline_runs
critic_pass_rate	insights passing critic / total insight attempts (v2)

14.2 SLOs (proposed)

SLO	Target	Alert
Gate pass rate	≥ 90% over 7d	Page if < 80% for 1h
PII leak eval	0 failures	Block deploy
Ingest success	≥ 99% (excl. DLQ)	Ticket if < 95% for 24h

14.3 Structured log schema

```
{
  "severity": "INFO",
  "run_id": "uuid",
  "node": "validate_load",
  "source": "demo",
  "rows_in": 21,
  "rows_silver": 17,
  "rows_quarantined": 4,
  "duration_ms": 842,
  "status": "ok"
}
```

14.4 Runbooks

Quarantine rate spiked (> 35%)

1. `SELECT error, COUNT(*) FROM quarantine_orders GROUP BY 1 ORDER BY 2 DESC`
2. Identify dominant error class ([ROW_VALIDATION](#) vs [DUPLICATE_ORDER_ID](#))
3. Fix upstream CSV or source schema; re-ingest

Quality gate blocked (freshness)

1. Check [gold_quality.last_ingest_at](#)
2. Confirm Scheduler / Pub/Sub trigger firing
3. Run manual `etl run --source inbox`

Critic violation loop

1. Inspect [critic.violations](#) in graph state
2. Compare against [get_gold_metrics](#) output

3. HITL approve corrected draft or fix gold SQL

15. Testing strategy

IMPLEMENTED

Status: IMPLEMENTED (v1) · SPECIFIED (v2 evals)

Layer	v1 IMPLEMENTED	v2 SPECIFIED
Unit	Pydantic SilverOrder , hash/idempotency	Presidio scanner, critic parser
Integration	test_pipeline , test_quality , test_http	LangGraph checkpoint resume
Eval	Quality gate threshold test	PII leak golden set, faithfulness set
E2E	CLI + Streamlit manual	GCS upload → Pub/Sub → Cloud Run

Current: 29/29 pytest passing. MVP gate: [./harness/e2e.sh](#)

Test file	Validates
test_pipeline.py	Idempotency, quarantine, gold KPIs
test_quality.py	Gate blocks when quarantine rate high
test_http.py	JSON extract, source registry
test_pii.py	PII redaction, no leak in output
test_critic.py	Faithfulness — rejects hallucinated numbers
test_gov_graph.py	FOIA graph E2E — status=complete
test_mcp_tools.py	Allowlist permit/deny
test_infra.py	GCP config, Pub/Sub decode, BQ SQL rewrite

16. Operational model

SPECIFIED

Status: SPECIFIED

Environment	Warehouse	Graph	MCP	Trigger
local	DuckDB file	SqliteSaver	stdio	etl run
staging	BigQuery dev	Cloud SQL	SSE	GCS test bucket
prod	BigQuery prod	Cloud SQL	SSE	GCS inbox + Scheduler

Release gates:

- v1: [uv run pytest](#) green
- v2: pytest + eval suite (PII leak, faithfulness, checkpoint resume) green before GCP promote

17. Implementation roadmap

Phase	Deliverable	Status
v1.0	Data plane, CLI, dashboard, tests	Done
v1.1	MCP server, 10 tools, Cursor registration	Next
v1.2	LangGraph, 8 nodes, SqliteSaver, critic	Next
v2.0	PII plane: Presidio, vault, redacted previews	Planned
v2.1	GCP: GCS, Pub/Sub, Cloud Run	Planned
v2.2	BigQuery, Cloud SQL checkpoints, Cloud Build	Planned
v2.3	Langfuse, eval suite, HITL dashboard	Planned

18. Conclusion

Operator ETL is an **agentic system that works** because agents have a narrow, enforced job: orchestrate decisions over a deterministic, tested data plane. MCP tools are the boundary. GCP provides durable storage, event-driven intake, and managed execution. PII and quality gates fail closed.

Ship now: FOIA MVP — [./scripts/demo_mvp.sh](#) — 12 comments, 10 silver, 2 quarantined, critic-verified insight.

Share externally: Clone <https://github.com/khaosans/operator-etl> and run `make e2e`. PDFs for decks: [docs/share/](#).

Build next: BigQuery gold dialect lift, HITL gov dashboard polish, Regulations.gov adapter.

Appendix A — Demo walkthrough

> Drop [samples/orders.csv](#) (21 rows). Seventeen pass [silverOrder](#) validation; four quarantine with explicit errors. Gold SQL: 17 orders, 10 customers, revenue 1373.82. Quality gate passes at 19.1%. Re-run same file: hash skip, zero duplicate rows. *(Verified: [tests/test_pipeline.py](#))*

Appendix B — MCP + GCP quick reference

Component	Local	GCP
Inbox	drops/inbox/	gs://operator-etl-inbox/
Warehouse	DuckDB	BigQuery
Checkpoints	SQLite	Cloud SQL
MCP	stdio	SSE (Cloud Run)
Secrets	.env	Secret Manager
Trigger	etl run	Pub/Sub + Scheduler

Appendix C — References

Proof matrix with test mapping: FOUNDATIONS.md

1. Databricks. (2020). Medallion Architecture. <https://www.databricks.com/glossary/medallion-architecture>
2. Kleppmann, M. (2017). **Designing Data-Intensive Applications**. O'Reilly. Ch. 11 — stream processing and idempotent consumers.
3. LangChain. LangGraph documentation. <https://langchain-ai.github.io/langgraph/>
4. Anthropic. Model Context Protocol specification. <https://modelcontextprotocol.io/>
5. NIST. (2010). SP 800-122 — Guide to Protecting the Confidentiality of Personally Identifiable Information. <https://csrc.nist.gov/publications/detail/sp/800-122/final>
6. Goodhart, C. A. E. (1975). Problems of Monetary Management: The U.K. Experience. In **Papers in Monetary Economics**. Reserve Bank of Australia.
7. Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. **NeurIPS**. <https://arxiv.org/abs/2005.11401>
8. NIST. (2023). AI Risk Management Framework (AI RMF 1.0). <https://www.nist.gov/itl/ai-risk-management-framework>
9. U.S. Congress. 5 U.S.C. § 552 — Freedom of Information Act. <https://www.foia.gov/>
10. OWASP. Top 10 for Large Language Model Applications. <https://owasp.org/www-project-top-10-for-large-language-model-applications/>

— End of white paper —